

**Technical University of Moldova**
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

# REPORT

Laboratory work No. 7

## Process Management, I/O Redirection, and IPC Pipes in Linux

Rotaru Anastasia, FAF-242

Course: **Operating Systems**

**Checked by:** Patricia Reițman, asist. univ.

Chişinău, 2026

# Contents

**1 Introduction** **2**

**2 Objectives** **2**

**3 Theoretical Background** **2**

**4 Task 1: Process Creation, Image Replacement and Waiting** **2**
4.1 Source code . . . . . 3
4.2 Execution output . . . . . 3
4.3 Observations . . . . . 3

**5 Task 2: A Zombie Process** **4**
5.1 Source code . . . . . 4
5.2 Execution output . . . . . 4
5.3 Observations . . . . . 4

**6 Task 3: Output Redirection** **6**
6.1 Source code . . . . . 6
6.2 Execution output . . . . . 7
6.3 Observations . . . . . 7

**7 Task 4: An Anonymous Pipe** **7**
7.1 Source code . . . . . 7
7.2 Execution output . . . . . 7
7.3 Observations . . . . . 9

**8 Conclusions** **9**

# 1 Introduction

Almost every useful action in a Unix shell depends on processes. When a user types `ls -l`, the shell does not list the files itself. It creates a new process, replaces that process with the `ls` program, and waits for it to finish. When a user types `ls -l > out.txt` or `ls /etc | wc -l`, the shell also changes where the output goes, either to a file or to another process through a pipe. These steps are carried out with a small set of system calls provided by the kernel.

This laboratory work makes these steps visible. Instead of using the shell, the work reproduces its behaviour in four short C programs that call the POSIX interface directly. Each program is small enough that every line can be linked to an observable result, either in the process table, in the contents of a file, or in the number printed on the terminal.

# 2 Objectives

The aim of this laboratory work is to understand how a Linux process is created, how it runs another program, how its standard output can be redirected to a file, and how two processes can communicate through an anonymous pipe. The work reproduces, in four short C programs, the operations that the shell performs for the commands `ls -l`, `ls -l > out.txt` and `ls /etc | wc -l`.

# 3 Theoretical Background

The function `fork()` creates a new process that is a copy of the caller. It returns the child's PID to the parent and the value 0 to the child, so the two processes can follow different branches of the same program. The function `execlp()` replaces the program image of the calling process with a new program, while the PID and the open file descriptors remain the same. A successful call to `exec` never returns, so any statement that follows it runs only when the call has failed.

A process that terminates keeps a small entry in the process table until its parent collects the exit status with `waitpid()`. Until then the process is a zombie, shown with the state `Z` by the `ps` utility. The zombie uses no memory and no processor time, but its entry remains allocated.

Every process has a descriptor table in which descriptors 0, 1 and 2 stand for standard input, standard output and standard error. The function `open()` always returns the lowest unused descriptor number. Therefore, if descriptor 1 is closed before a file is opened, the file becomes standard output. A pipe created by `pipe()` has a read end and a write end. The reader receives end-of-file only when every copy of the write end has been closed, including the copies held by the parent.

# 4 Task 1: Process Creation, Image Replacement and Waiting

The program `run.c` performs the three steps that the shell performs for the command `ls -l`. The process copies itself with `fork()`, the child becomes `ls` with `execlp()`, and the parent waits for the child with `waitpid()`.

## 4.1 Source code

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command: ls -l */
int main(void){
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);
    pid_t pid = fork();                  /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {                      /* only the child runs this */
        printf("child: my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);   /* step 2: become ls */
        perror("exec");                  /* runs only if exec failed */
        exit(127);
    }
    int status;
    waitpid(pid, &status, 0);            /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Figure 1: Source code of run.c

The `fflush` calls make sure that the text written before `fork()` is not duplicated in the child, because the buffer of standard output is copied together with the rest of the process memory. The variant with the command `date` replaces only the line that calls `execlp`, and the variant with the name `nosuchprog` is used to observe the error path.

## 4.2 Execution output

The outputs below are the expected results from the laboratory handout. Replace them with the output of your own runs before submitting.

## 4.3 Observations

**Question 1. Why is the line `perror("exec")` printed only with `nosuchprog`?** When `execlp("ls", ...)` succeeds, the memory of the child is replaced by the code of `ls`, so the instruction that calls `perror` no longer exists in that process. With `nosuchprog`

```
parent: my PID is 15
child: my PID is 16, my parent is 15
total 20
-rwxr-xr-x 1 root root 16136 Oct 18 20:25 run
-rwxr-xr-x 1 root root   911 Oct 18 20:25 run.c
parent: child 16 ended with status 0
```

Figure 2: Expected output of run with ls -l

```
exec: No such file or directory
parent: child 32 ended with status 127
```

Figure 3: Output with a nonexistent program

no program can be found, `execlp` returns $-1$, and the child continues to the next line, which prints the error and exits with status 127.

**Question 2. Which number does the child print as its parent, and whose PID is it?** The child calls `getppid()`, which returns the PID of the process that created it. The number is therefore the PID printed by the parent on its first line, which is 15 in the expected output above.

**Question 3. What status does the parent see for `ls`, and what status for `nosuchprog`?** For `ls` the child exits normally and the parent reads the status 0 with `WEXITSTATUS`. For `nosuchprog` the child exits with the code 127, which is the conventional code for a command that cannot be found, and the parent reads 127.

# 5 Task 2: A Zombie Process

The program `zombie.c` is the same as `run.c`, with one added line, `sleep(20);`, placed before `waitpid()`. While the parent sleeps, the child has finished running `ls` but has not been collected, so it remains in the process table as a zombie.

## 5.1 Source code

## 5.2 Execution output

The listing below is the expected output from the handout. The process table is inspected with the command `ps -o pid,ppid,stat,cmd`.

## 5.3 Observations

**Question 1. Which letter does `ls` have in the STAT column, and what does it mean?** The child is shown with the letter `Z`. This means that the process has terminated but has not been reaped by its parent. The kernel keeps only its entry in the process table, so that the exit status can be delivered when the parent calls `waitpid()`.

**Question 2. After 20 seconds, is the line `[ls] <defunct>` still there? Which line of the code removed it?** After 20 seconds the line is no longer shown. The line

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the same program as run.c, with a delay before waitpid */
int main(void){
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }
    int status;
    sleep(20);                              /* the new line */
    waitpid(pid, &status, 0);
    return 0;
}
```

Figure 4: Source code of zombie.c

```
 PID  PPID  STAT  CMD
  39     1  S     ./zombie
  41    39  Z     [ls] <defunct>
  42     1  R     ps -o pid,ppid,stat,cmd
```

Figure 5: Expected output of ps during the delay

`waitpid(pid, &status, 0);` collected the exit status of the child, and after this call the kernel removed the remaining entry from the process table.

# 6 Task 3: Output Redirection

The program `redirect.c` does what the shell does for the command `ls -l > out.txt`. The child closes descriptor 1, which is standard output, and then opens `out.txt`. Because descriptor 1 is the lowest free number, the file receives it. The child then runs `ls`, which writes to descriptor 1 as it always does, so its listing goes to the file without any change to `ls`.

## 6.1 Source code

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

/* does the same as the shell command: ls -l > out.txt */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);                       /* descriptor 1 is now free */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);   /* ls writes to 1, as always */
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

Figure 6: Source code of redirect.c

The message about the descriptor is written to standard error, not to standard output. Standard error still refers to the terminal, so the message is visible even though standard output has been redirected.

## 6.2 Execution output

The output below is the expected result from the handout. Replace it with the output of your own run, including the result of `head -3 out.txt`.

```
open gave descriptor 1
done: the listing is in out.txt
total 100
-rw-r--r-- 1 root root     0 Oct 18 20:25 out.txt
-rwxr-xr-x 1 root root 16136 Oct 18 20:25 redirect
```

Figure 7: Expected output of redirect

## 6.3 Observations

**Question 1. Which number did `open` return, and why exactly that one?** The function `open` returned 1. Descriptor 1 was freed by `close(1)`, and the kernel always assigns the lowest unused descriptor number to a newly opened file.

**Question 2. The listing of `ls` does not appear on the screen. Why?** The program `ls` was not changed. It writes to descriptor 1, but after the redirection that descriptor refers to `out.txt` and no longer refers to the terminal. Only the messages written to descriptor 2 still appear on the screen.

**Question 3. To redirect errors instead, which descriptor would be closed before `open`?** Descriptor 2, which is standard error, would be closed before calling `open`. This is the program-level equivalent of the shell notation `2> err.txt`.

# 7 Task 4: An Anonymous Pipe

The program `pipeline.c` builds the shell pipeline `ls /etc | wc -l` by hand. The parent creates one pipe and forks two children. The first child makes its standard output the write end of the pipe and runs `ls /etc`. The second child makes its standard input the read end of the pipe and runs `wc -l`. The function `dup` copies a descriptor onto the smallest free number, so descriptor 1 of the first child and descriptor 0 of the second child become the two ends of the pipe.

## 7.1 Source code

## 7.2 Execution output

The result of the hand-built pipeline must be compared with the result of the shell pipeline. The outputs below are the expected results from the handout. Replace them with the results of your own runs.

To observe the deadlock, the line `close(p[1]);` in the parent is commented out and the program is run again. The program does not finish and is stopped with Ctrl-C.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command: ls /etc | wc -l */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                  /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);                      /* smallest free number: 1 */
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {                  /* wc: its 0 becomes p[0] */
        close(0);
        dup(p[0]);                      /* smallest free number: 0 */
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p[0]);                        /* the parent uses neither end */
    close(p[1]);

    wait(NULL);
    wait(NULL);
    return 0;
}
```

Figure 8: Source code of pipeline.c

```
$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
238
$ ls /etc | wc -l
238
```

Figure 9: Comparison with the shell pipeline

```
$ ./pipeline
(the program does not end)
^C
```

Figure 10: Program with the parent write end left open

### 7.3 Observations

**Question 1. Did `./pipeline` and `ls /etc | wc -l` give the same number?** Yes. Both commands count the same entries in `/etc`, so the hand-built pipeline reproduces the behaviour of the shell.

**Question 2. After the change, which of the two children never ends, `ls` or `wc`?** The child that runs `wc` never ends. The child that runs `ls` finishes normally, because it terminates after writing its output.

**Question 3. What is `wc` waiting for?** The program `wc` waits for end-of-file on its standard input. End-of-file is delivered only when every copy of the write end of the pipe has been closed. The parent still holds a copy of the write end, so the kernel never sends end-of-file, and `wc` waits forever.

## 8 Conclusions

The laboratory work showed that the commands of the shell are built from a small set of system calls. The sequence `fork()`, `execlp()` and `waitpid()` runs a program and collects its exit status, and a parent that does not wait leaves a zombie in the process table. Output redirection is obtained by changing the descriptor table of the child before the program runs, because `open()` returns the lowest free descriptor. A pipeline uses the same mechanism together with `pipe()` and `dup()`, and it requires every process to close the pipe ends it does not use, since a single open write end prevents end-of-file from reaching the reader. The results of the hand-built programs matched the corresponding shell commands.